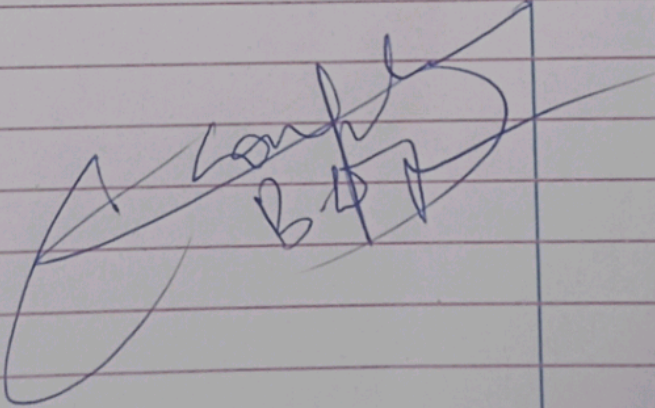


Index

Name : RAGUL A Std. 3rd year Sec. -

Roll No. 230701252 Sub: Mobile Application Development
Laboratory

S.No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.		Kotlin Basics		
2.		Functions		
3.		Classes & Objects		
4.		Build First Android App		
5.		Layouts		
6.		App Architecture		
		Persistence		
7.		Advanced RecyclerView		
		Use Cases		
8.		Connect to the Internet		
9.		Repository pattern & work managers		
10.		App UI Design		



Teacher's Signature

1. KOTLIN BASICS

Aim:

To understand the basic syntax and features of the Kotlin programming language.

Algorithm:

- * Start the Android application
- * Create a new android project in android studio using Kotlin and Jetpack compose.
- * Import the required packages and libraries.
- * Apply the Application theme.
- * Run the application in Android or physical device.
- * Verify output.
- * Stop the program.

Result:

The basic Kotlin program was implemented successfully and the fundamental syntax and operation were understood.

3:43



LAB 1

Build First Android App

Enter first text

aabc



Enter second text

aabc

Check Match

Correct

2. Functions

Aim:

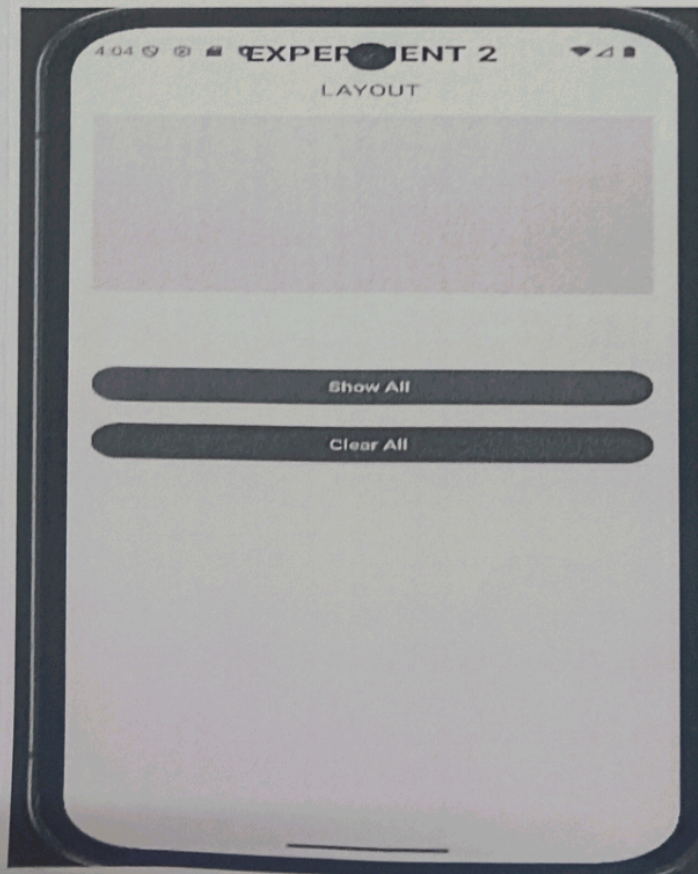
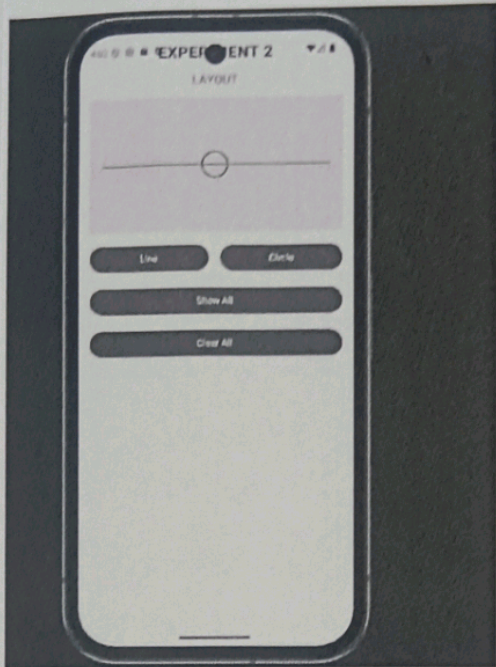
To study and implement functions in Kotlin for performing specific tasks.

Algorithm:

- * Start the Application.
- * Create a new Android project in Android studio using Kotlin and Jetpack Compose.
- * Import the required packages and libraries.
- * Apply the application theme.
- * Implement the required UI functionality using state variables and event handlers.
- * Run the application in Android emulator.
- * Verify output.

Result:

Function were created and executed successfully demonstrating modular programming in Kotlin.



3. classes and objects

Aim:

To understand object oriented programming concepts using classes and objects in Kotlin.

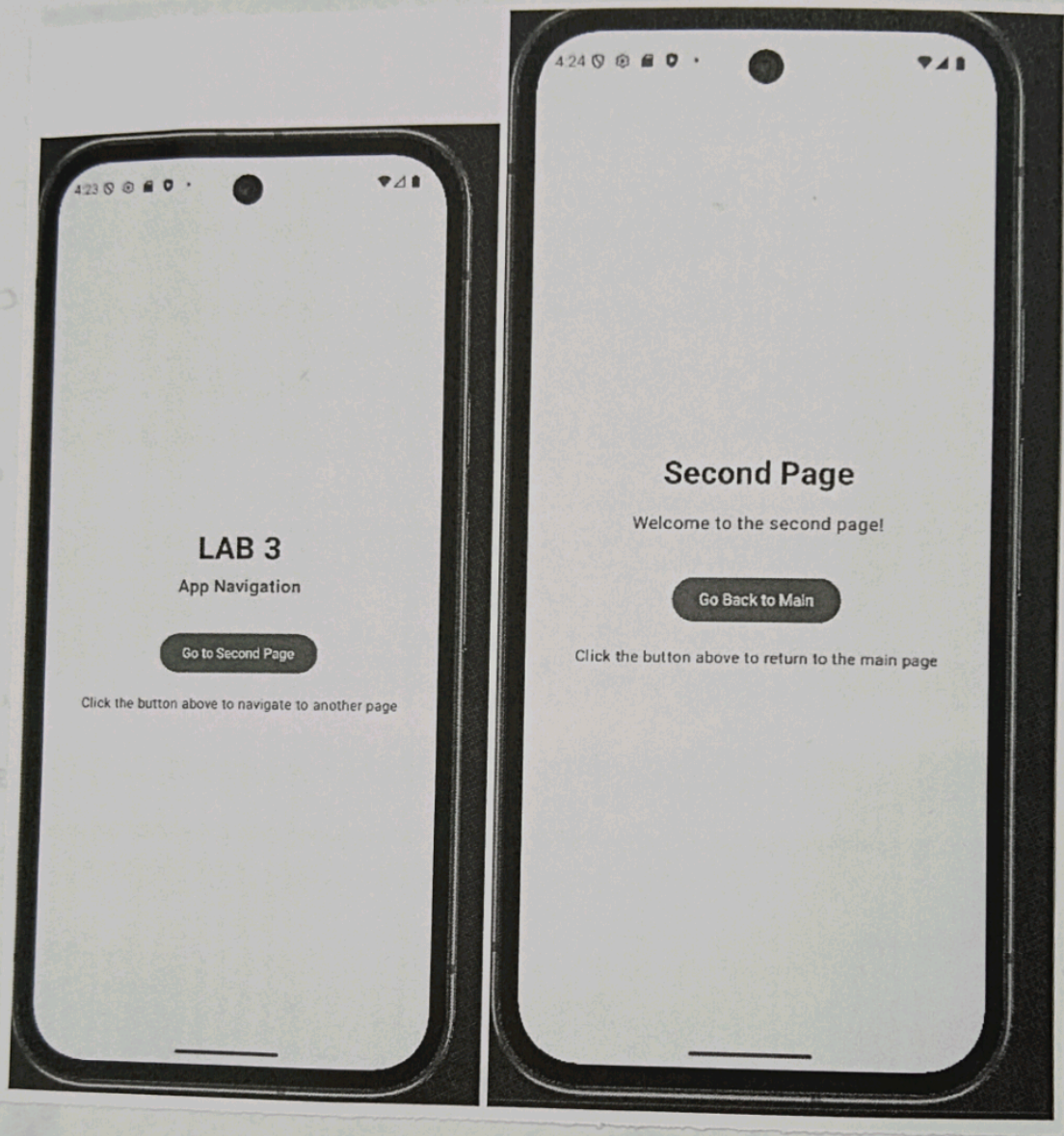
Algorithm:

- * Start the Android application.
- * Create a new Android project in Android studio using Kotlin & setpack compose.
- * Import required packages and libraries.
- * Apply application theme.
- * Run the application in an Android emulator or physical device.
- * Verify the output.
- * Stop the program.

PO

Result:

Classes and objects were created successfully and the concept of object oriented programming was understood.



4. Build First Android App

Aim:

To create and run a simple Android application using Android studio and Kotlin.

Algorithm:

- * Start the Android application.
- * Create a new project in Android studio using Kotlin and Setback compose.
- * Import required package and libraries.
- * Apply the application theme.
- * Run the application in an Android emulator or physical device.
- * Verify the output.
- * Stop the program.

Result:

A basic Android application was created and executed successfully in emulator/device.

5. LAYOUTS

Aim:

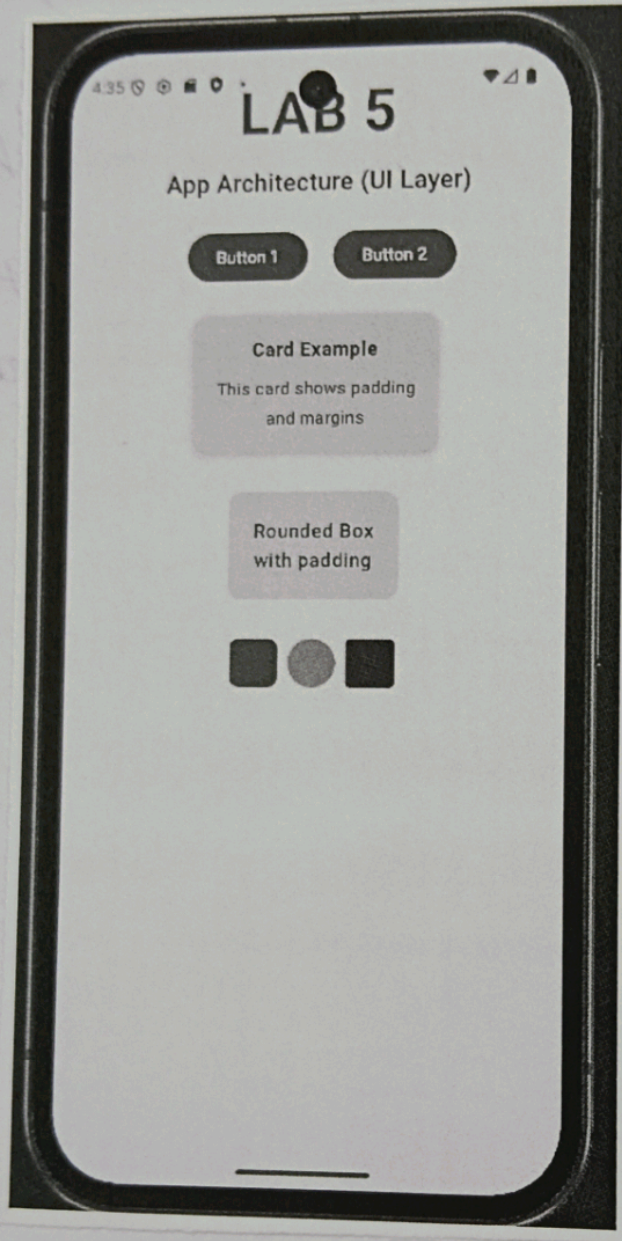
To design and implement different layouts in an android application using Setpack Compose.

Algorithm:

- * Start the Android application.
- * Create a new Android project in Android studio using Kotlin & Setpack Compose.
- * Import the required packages and libraries.
- * Apply the application theme.
- * Implement the required UI functionality using state variables and event handlers.
- * Run the application in an android emulator.
- * Verify output.
- * Stop program.

By Result:

Layout were successfully designed and components were arranged properly in the application.



6. App Architecture Persistence

Aim:

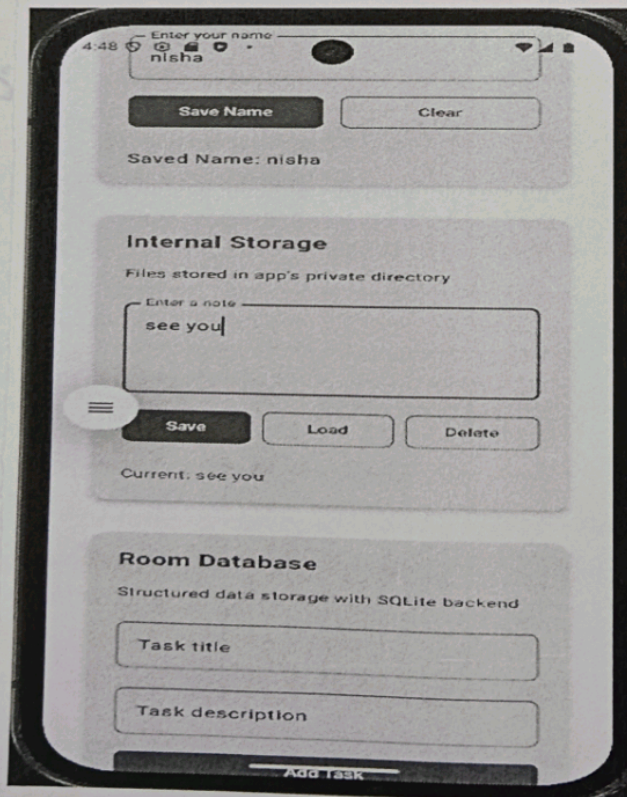
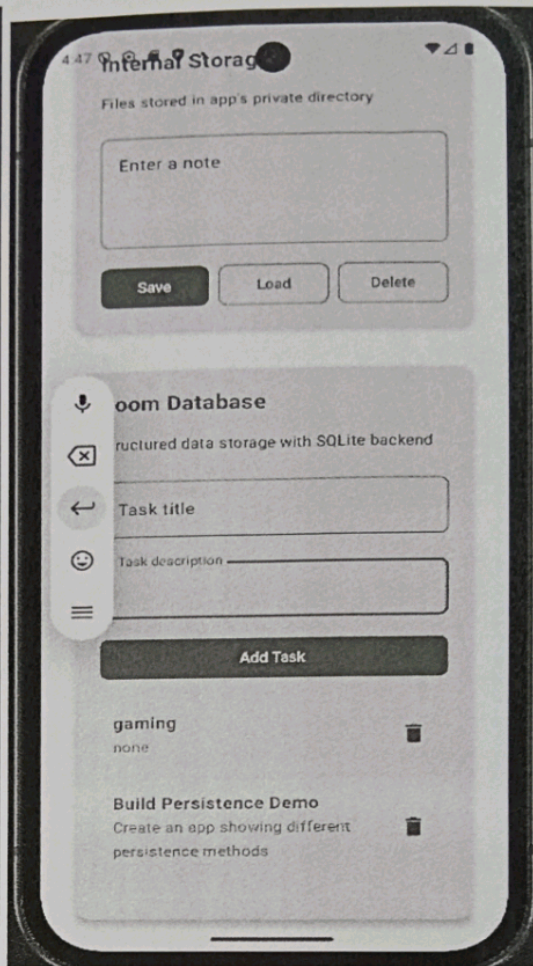
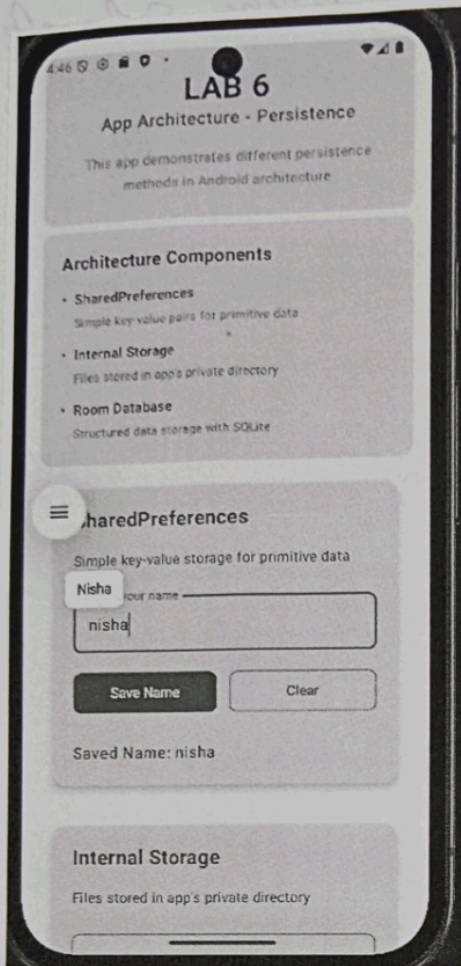
To implement app Architecture between different screens in an android application.

Algorithm:

- * Start the android application.
- * Open android studio and create a new project using Kotlin & Setpack Compose.
- * Add navigation compose dependency in project.
- * Create different screens.
- * Initialize a nav controller to manage navigation between screens.
- * Define nav and specify routes for each screen.
- * Run the application on emulator.
- * Verify output.
- * Stop program.

Result:

App Architecture persistence between multiple screens was implemented successfully.



7. Advanced RecyclerView use cases

Aim:

To study the Advanced RecyclerView use cases in ~~Android~~ Android.

Algorithm:

* Start the Android application and open Android studio to create a new Kotlin project.

* Add the recyclerView dependency & design a layout containing a recyclerView.

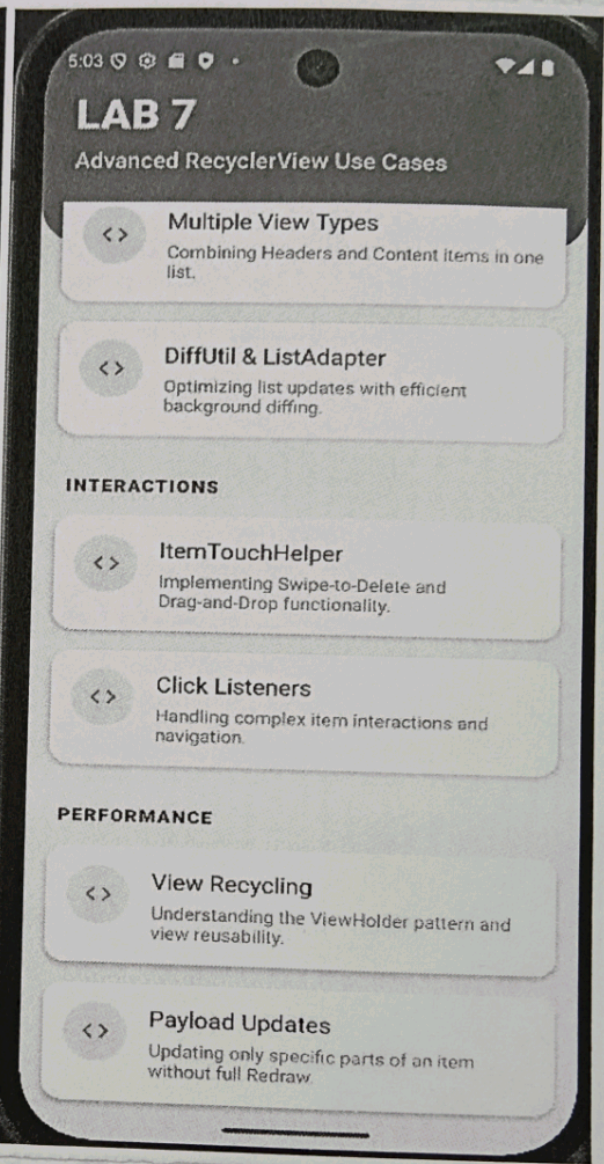
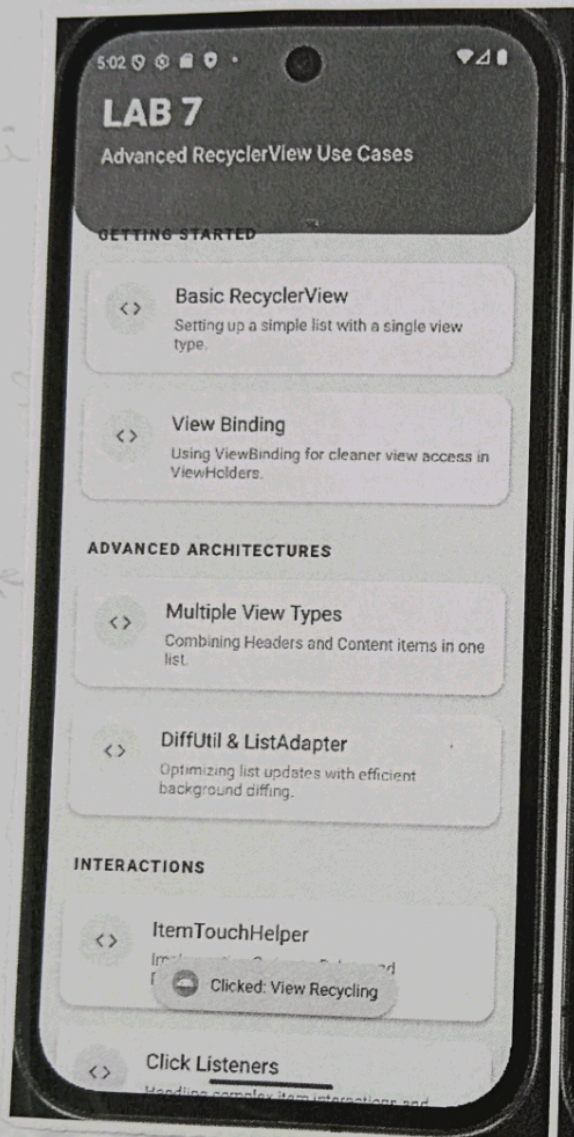
* Create a data model and an adapter with view Holder to display the list items.

* Add item ⁿinteractions such as click listeners and swipe actions using itemTouch helpers.

* Run the application & verify output.

Result:

Advanced RecyclerView use cases was observed and understood successfully.



8. Connect to the Internet

Aim:

To connect the Android application to the internet & retrieve data from online sources.

Algorithm:

- * To connect the Android application to the internet and retrieve.

- * Start the android application & create a new project in Android studio using Kotlin.

- * Add the required internet & network permissions in the file.

- * Use connectivity manager to monitor the network connection status.

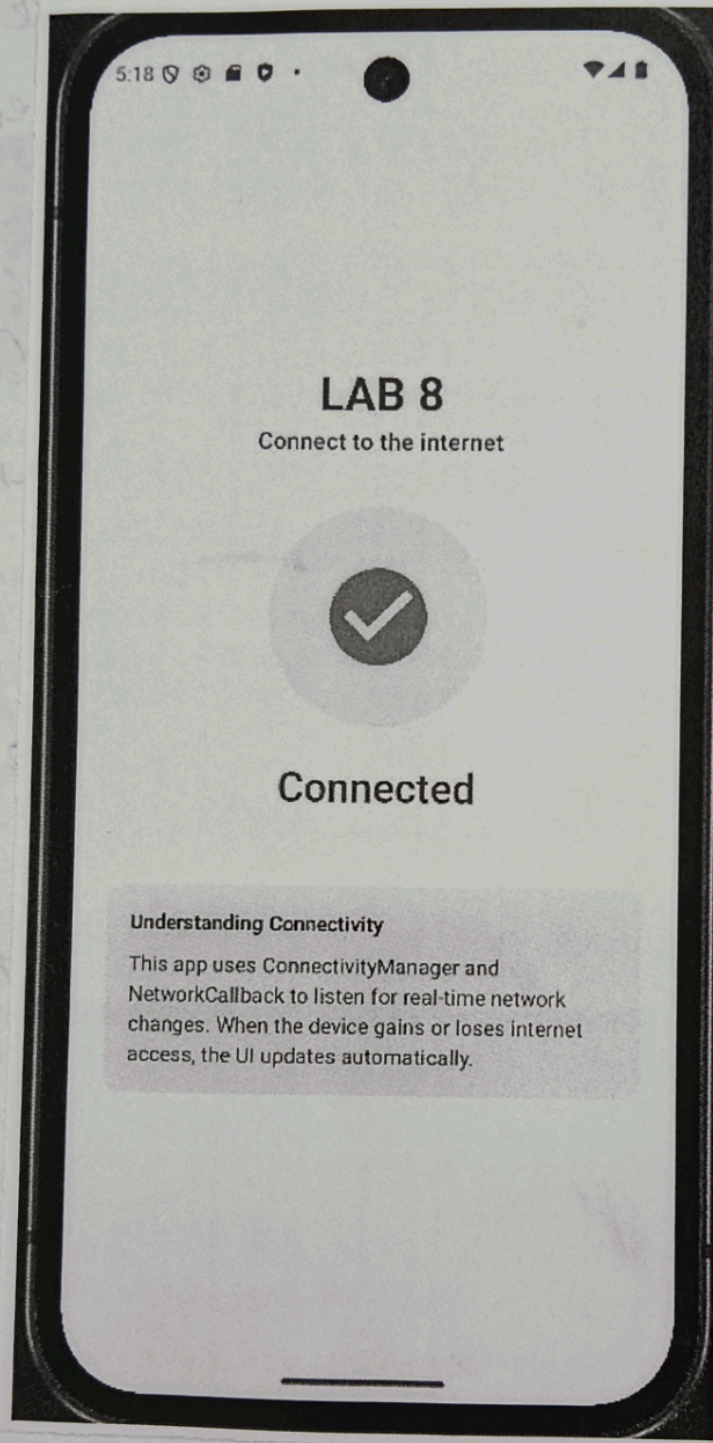
- * Implement network callback to detect when the device connects or disconnects from internet.

- * Update the UI to display the connection status.

- * Run the application & verify the status change.

Result:

The application successfully connected to internet & receive required data.



LAB 8

Connect to the internet



Connected

Understanding Connectivity

This app uses `ConnectivityManager` and `NetworkCallback` to listen for real-time network changes. When the device gains or loses internet access, the UI updates automatically.

9. Repository pattern and Workmanagers

Aim:

To implement the repository pattern and workmanagers for background tasks in Android.

Algorithm:

- * Start the Android application & create a new project in Android studio using Kotlin.

- * Create a Room database & implement the repository pattern to manage data operations.

- * Create a workclass, using workmanagers to perform background tasks such as syncing data.

- * Connect the repository with worker to store and update data in database.

- * Run the application & verify.

Result:

Repository pattern & workmanagers were implemented successfully for data management & background processing.

5:25

LAB 9

Repository Pattern and Work Manager

How it works:

1. Repository manages data from Room DB.
2. Work Manager runs SyncWorker in background.
3. UI updates automatically via Flows.

No goals found. Click 'Sync Now' to start!

+ Sync Now

10. App UI Design

Aim:

To design an attractive & user-friendly interface for an Android application.

Algorithm:

* Start the Android application & create a new project in Android studio using Kotlin.

* Design the main UI layout using components such as button, text.

* Implement UI design principles.

* Add interactive components.

* Apply proper alignment.

* Run the application.

Result:

The application UI was designed successfully with proper layout & studying

